



SISTEMA DE RECOMENDACIÓN · MOVIELENS 25M

Informe técnico de cambios

OmniRec-Movies

Fase 4 — MLOps + Recuperación semántica (RAG) y dockerización

Proyecto	Sistema de recomendación de películas (MovieLens 25M)
Grupo	3 — Vincit Data
Asignatura	Machine Learning 2026-I
Metodología	CRISP-DM
Alcance	Búsqueda semántica + pipeline reproducible MLOps + Docker
Fecha	21 de junio de 2026

Documento de registro detallado de todas las modificaciones realizadas: nuevos módulos, integración en la plataforma, decisiones técnicas, dockerización, resolución de incidencias y validación extremo a extremo.

Contenido

1. Resumen ejecutivo	2
2. Contexto y punto de partida	3
3. Componente de recuperación semántica (RAG)	3
3.1. Descriptores textuales (src/descriptors.py)	3
3.2. Embeddings (src/embeddings.py)	3
3.3. Índice vectorial FAISS (src/index.py)	4
3.4. Búsqueda con re-ranking y evidencia (src/search.py)	4
3.5. Capa generativa (src/generate.py)	4
4. Pipeline y andamiaje MLOps	4
4.1. Configuración declarativa (config/)	4
4.2. Orquestador (src/pipeline.py)	5
4.3. Tracking de experimentos (src/tracking.py)	5
4.4. Versionado de artefactos (src/registry.py)	5
4.5. Evaluación (src/evaluate.py)	5
4.6. Monitoreo y reentrenamiento (src/monitor.py)	5
4.7. Entrenamiento y baseline (src/train.py, src/baseline.py)	5
5. Integración en la plataforma	5
5.1. Backend (FastAPI)	5
5.2. Frontend (Next.js “OmniCine”)	6
5.3. Notebook e informes	6
6. Decisiones técnicas y problemas resueltos	6
7. Dockerización completa	7
7.1. Servicios	7
7.2. Archivos creados	7
7.3. Estrategia de artefactos: <i>build-on-first-run</i>	7
7.4. Frontend liviano	8
8. Completar el recomendador en Docker	8
9. Validación extremo a extremo con Docker	8
10. Diagnóstico de MLflow	9
11. Higiene del repositorio: .gitignore	9
12. Inventario completo de archivos	9
12.1. Archivos creados	9
12.2. Archivos modificados	10
13. Resultados y figuras	11
14. Cómo ejecutar	11
14.1. Con Docker (recomendado)	11
14.2. Nativo (sin Docker)	12
15. Conclusiones y trabajo futuro	12

1 Resumen ejecutivo

Este documento detalla la totalidad de los cambios incorporados al proyecto **OmniRec-Movies** durante la implementación de la **Fase 4** de la guía integradora (**MLOps + RAG / recuperación semántica**) y su posterior **dockerización** y validación.

El trabajo añade al recomendador colaborativo existente (notebooks 01-04) dos capacidades nuevas y un andamiaje operativo:

1. **Recuperación semántica multilingüe.** Cada película del catálogo (55 113 títulos) se describe con texto (título + géneros + *tags* genome más relevantes), se proyecta con un modelo *Sentence-Transformers* preentrenado y se indexa con **FAISS**. Una consulta en lenguaje natural —por ejemplo “*thriller psicológico sobre la memoria*”— recupera *Memento* y *Eternal Sunshine of the Spotless Mind*, exponiendo los *tags* que justifican cada resultado.
2. **Capa RAG generativa.** Sobre la recuperación, un modelo de lenguaje (Claude, en inferencia) redacta una recomendación fundamentada citando solo las películas recuperadas, con *fallback* determinista por plantilla.
3. **MLOps mínimo viable.** Pipeline reproducible por etapas, registro de experimentos con **MLflow**, versionado de artefactos con manifiestos **SHA256** y monitoreo de calidad / *drift* con política de reentrenamiento.
4. **Dockerización completa.** Cuatro servicios (*backend*, *web*, *MLflow*, *Jupyter*) orquestados con Docker Compose, de modo que `git clone + docker compose up --build` deja todo funcional en cualquier computadora.

Indicadores principales

0.88 Recuperación P@10 Solapamiento de géneros sobre 6 consultas	25.1 % Cobertura genome Fracción del catálogo con <i>tags</i> genome	0.882 / 0.663 RMSE / MAE Folding-in, leave-last-one-out temporal
0.426 / 0.249 HitRate@10 / NDCG@10 Leave-one-out vs. 99 negativos	55 113 × 384 Índice vectorial FAISS IndexFlatIP (exacto)	16 Módulos nuevos en <i>src</i> / Pipeline MLOps + RAG

i En una frase

Un recomendador colaborativo ya existente se amplió con **búsqueda en lenguaje natural explicable** y **RAG generativo**, se rodeó de un **flujo MLOps reproducible** y se empaquetó en **Docker** para arrancar con un solo comando.

2 Contexto y punto de partida

El repositorio ya contaba con un pipeline de Machine Learning clásico (notebooks 01-03), un **backend FastAPI** y un **frontend Next.js** (“OmniCine”). Sin embargo, los directorios `src/`, `config/` y `models/` estaban prácticamente vacíos (solo READMEs), el notebook `05_Semantic_Search_RAG.ipynb` era un *placeholder*, y no existía ningún componente de recuperación semántica ni andamiaje de MLOps.

Los datos disponibles y versionados que sirvieron de insumo fueron los *parquets* intermedios: catálogo de películas, *genome-scores* (15,5 millones de filas de relevancia *tag*-película), diccionario de 1 128 *tags* genome, y una muestra de *ratings*.

i Qué significa “recuperación semántica (RAG)”

En lugar de buscar por coincidencia exacta de palabras, el sistema convierte cada película y cada consulta en un *vector* numérico que captura su *significado*. Películas con significado parecido quedan cerca en ese espacio, de modo que “thriller sobre la memoria” encuentra *Memento* aunque esa frase no aparezca en su ficha. El patrón **RAG** (*Retrieval-Augmented Generation*) añade un paso final: un modelo de lenguaje redacta la recomendación apoyándose solo en lo recuperado.

3 Componente de recuperación semántica (RAG)

Se implementó en el paquete `src/` y se integró en la plataforma. El flujo sigue el patrón canónico **recuperar** → **aumentar** → **generar**.



3.1 Descriptores textuales (`src/descriptors.py`)

Para cada película se construye un *descriptor* legible que concatena el título limpio, el año, los géneros y los *tags* genome más relevantes (top-12, relevancia $\geq 0,5$):

```
"The Matrix (1999). Géneros: Action, Sci-Fi.
Temas: virtual reality, dystopia, artificial intelligence, philosophy, ..."
```

La agregación de los 15,5 M de *genome-scores* se realiza con **Polars** (rápida y perezosa). Las películas sin genome (*long tail*) reciben un descriptor degradado (solo título + géneros); esta **cobertura parcial del 25.1 %** se cuantifica y se monitorea. Además se incorpora el número de *ratings* por película (`n_ratings`) para el re-ranking posterior.

3.2 Embeddings (`src/embeddings.py`)

Encoder **paraphrase-multilingual-MiniLM-L12-v2** (384 dimensiones, CPU), preentrenado y usado en *modo inferencia*, cumpliendo la política de la asignatura sobre *Transformers*. Es **multilingüe**: permite consultas en español sobre descriptores en inglés, que es el idioma de la plataforma. Los vectores se normalizan L2 (producto interno $\equiv$ coseno).

3.3 Índice vectorial FAISS (src/index.py)

Se construye un índice IndexFlatIP (exacto) de $55\,113 \times 384$. Se persisten:

- > `embeddings.npy` — los vectores del catálogo.
- > `index.faiss` — el índice de búsqueda.
- > `meta.parquet` — metadatos por película.
- > `tag_embeddings.npy` — embeddings de los 1 128 tags (para la evidencia).
- > `manifest.json` — versión, encoder, dimensiones, cobertura y **hash SHA256** de los embeddings.

Incluye *fallback* a NumPy si FAISS no está disponible.

3.4 Búsqueda con re-ranking y evidencia (src/search.py)

La recuperación combina la similitud semántica con un **re-ranking por prior de popularidad bayesiana**, que vincula la búsqueda con la señal del recomendador y prioriza títulos conocidos frente a homónimos de la *long tail*:

```
score = similitud_coseno + alpha * popularidad_log_normalizada # alpha = 0.25
```

Por cada resultado se calcula la **evidencia de pertinencia**: los *tags* de la película se re-ordenan por similitud con la consulta (usando los embeddings de *tags*) y se devuelven los más afines, de modo que la recuperación *no sea una caja negra* y el usuario entienda por qué se recomendó cada título.

3.5 Capa generativa (src/generate.py)

Construye un *prompt* fundamentado con la consulta y las fichas recuperadas, y un LLM (Claude, modelo `claude-haiku-4-5` por defecto) redacta la recomendación **citando solo las películas recuperadas** (instrucción anti-alucinación). Si no hay `ANTHROPIC_API_KEY`, una **plantilla determinista** produce la respuesta a partir de los resultados, de manera que la demo y la reproducibilidad nunca dependan de la red.

4 Pipeline y andamiaje MLOps

4.1 Configuración declarativa (config/)

Tres archivos YAML separan los parámetros del código:

- > `config/pipeline.yaml` — semilla, rutas, fuentes de datos, hiperparámetros del SVD, protocolo de evaluación y experimento de MLflow.
- > `config/rag.yaml` — encoder, parámetros de los descriptores, backend del índice, re-ranking y capa generativa.
- > `config/monitoring.yaml` — umbrales de calidad / *drift* y política de reentrenamiento.

4.2 Orquestador (src/pipeline.py)

CLI por etapas idempotentes:

```
python -m src.pipeline all          # prepare -> train -> baseline -> index ->
    evaluate -> register
python -m src.pipeline monitor      # reporte de calidad/drift + disparadores
```

4.3 Tracking de experimentos (src/tracking.py)

Envoltorio fino sobre **MLflow** (backend SQLite local). Registra parámetros, métricas y *tags* de cada corrida; degrada a un contexto *no-op* si MLflow no está instalado. Honra la variable de entorno `MLFLOW_TRACKING_URI` (clave para Docker).

4.4 Versionado de artefactos (src/registry.py)

Cada artefacto se versiona en `models/registry/<artefacto>/<version>/` con un **manifiesto** que registra versión, **SHA256**, versión del dataset, métricas y versión padre. Un alias `current.json` marca la versión en producción.

4.5 Evaluación (src/evaluate.py)

- **Recomendador:** ejercita el artefacto servido vía *folding-in* (el mismo mecanismo del backend) con split temporal *leave-last-one-out*; computa RMSE/MAE y HitRate@K / NDCG@K contra negativos muestreados.
- **Recuperación:** precisión por solapamiento de géneros sobre consultas etiquetadas + cobertura del índice.

4.6 Monitoreo y reentrenamiento (src/monitor.py)

Produce un reporte JSON con métricas de calidad, *drift* temporal de la distribución de *ratings* (PSI), cobertura, frescura de datos y degradación de NDCG@10 frente a la versión `current`. Evalúa la política de reentrenamiento y emite alertas.

4.7 Entrenamiento y baseline (src/train.py, src/baseline.py)

`train.py` reentrena el SVD ganador (hiperparámetros del notebook 03) y los clústeres KMeans, produciendo `svd_model.pkl`. `baseline.py` (añadido para completar el recomendador en Docker) reproduce la **popularidad bayesiana** del notebook 03:

```
C = media_global_de_ratings
m = percentil_60(n_votos_por_pelicula)
score = (v / (v + m)) * media_pelicula + (m / (v + m)) * C
```

5 Integración en la plataforma

5.1 Backend (FastAPI)

- `app/backend/ml/semantic.py`  **nuevo** — puente que carga el índice y expone la recupera-

ción y la generación, resolviendo el paquete `src` vía `sys.path`.

- > `app/backend/routers/search.py`  nuevo — endpoint `GET /api/search/semantic` (parámetros `q`, `k`, `generate`), independiente del recomendador.
- > `app/backend/main.py`  modificado — registra el router de búsqueda y hace el arranque **resiliente** (si faltan artefactos del recomendador, la búsqueda sigue disponible).
- > `app/backend/ml/engine.py`  modificado — tolerante a la ausencia de `links.csv` (los pósters caen a arte procedural; el recomendador funciona).

5.2 Frontend (Next.js “OmniCine”)

- > `app/web-app/app/buscar/page.tsx`  nuevo — página de búsqueda semántica: barra en lenguaje natural, consultas de ejemplo, *toggle* de “Respuesta IA” y grilla de resultados.
- > `app/web-app/components/SearchResultCard.tsx`  nuevo — tarjeta a medida que destaca la evidencia (*tags*) y la afinidad.
- > `lib/types.ts`, `lib/api.ts`  modificados — tipos `SemanticResult` / `SemanticResponse` y método `semanticSearch`.
- > `components/Navbar.tsx`  modificado — enlace “Buscar” y lupa redirigida a `/buscar`.
- > `next.config.ts`  modificado — `output: "standalone"` para una imagen Docker liviana.

5.3 Notebook e informes

Se reescribió y **ejecutó** el notebook `05_Semantic_Search_RAG.ipynb` (descriptores, embeddings, visualización t-SNE, 6 consultas documentadas, generación, evaluación y narrativa MLOps), y se redactaron `reports/PLAN_FASE4_MLOPS_RAG.md` e `reports/INFORME_FASE4_MLOPS_RAG.md`.

6 Decisiones técnicas y problemas resueltos

Tema	Decisión / resolución
Encoder de embeddings	Se pasó de <code>all-MiniLM-L6-v2</code> (inglés) a <code>paraphrase-multilingual-MiniLM-L12-v2</code> porque las consultas de OmniCine son en español; resolvió que se devolvieran homónimos por coincidencia léxica.
Re-ranking por popularidad	Sin él, las consultas recuperaban títulos oscuros de la <i>long tail</i> (p. ej. “Opera Jawa” para <i>space opera</i>). El prior de popularidad bayesiana prioriza títulos canónicos y con <i>genome</i> .
torch en Windows	La versión 2.12 fallaba al cargar (<code>WinError 1114</code> en <code>c10.dll</code>). Se fijó <code>torch==2.4.*</code> , estable y compatible con <code>numpy<2</code> . En Docker (Linux) el problema desaparece.
MLflow 3.x	El <i>file store</i> fue deshabilitado; se migró a backend SQLite . El <i>wrapper</i> ahora honra <code>MLFLOW_TRACKING_URI</code> .
Nombres de métricas MLflow	MLflow no admite “@”; se sanea <code>ndcg@10</code> → <code>ndcg_at_10</code> .

Tema	Decisión / resolución
Codificación UTF-8 en Windows	src/utils.py reconfigura stdout a UTF-8 para evitar UnicodeEncodeError con acentos y nombres de tags.

7 Dockerización completa

i Objetivo

`git clone` + `docker compose up --build` y todo funcional, en cualquier sistema operativo.

7.1 Servicios

Servicio	Imagen	Puerto	Rol
backend	Python 3.12 + ML	8000	FastAPI: recomendador + búsqueda semántica
web	Node 20 (standalone)	3000	OmniCine, incluye <code>/buscar</code>
mlflow	(imagen backend)	5000	UI de experimentos (perfil <code>tools</code>)
jupyter	(imagen backend)	8888	Notebooks 01-05 (perfil <code>notebooks</code>)

Cuadro 2: Servicios definidos en `docker-compose.yml`.

7.2 Archivos creados

Se añadieron `docker-compose.yml`, `app/backend/Dockerfile`, `app/backend/docker-entrypoint.sh` y `app/web-app/Dockerfile`; los contextos de build `.dockerignore` (raíz) y `app/web-app/.dockerignore`, el `.env.example` y el `Makefile`; además de una sección “Ejecución con Docker” en el README.

7.3 Estrategia de artefactos: *build-on-first-run*

El modelo Sentence-Transformers se **hornea en la imagen** durante el `docker build` (funciona offline en runtime). El índice, el SVD y el baseline se generan en el **primer arranque** mediante el `entrypoint` y se persisten en un volumen nombrado:

```
if [ "$ENSURE_ARTIFACTS" = "1" ]; then
  [ -f models/rag_index/manifest.json ] || { python -m src.pipeline prepare;
  \
                                          python -m src.pipeline index;
  }
  [ -f models/svd_model.pkl ] || python -m src.pipeline train
  [ -f models/baseline_scores.pkl ] || python -m src.pipeline baseline
fi
exec "$@"
```

Así el primer `up` tarda algunos minutos y los siguientes arrancan al instante. Los volúmenes `models`, `mlruns` y `appdata` persisten los artefactos, los experimentos y la base de datos de la aplicación.

7.4 Frontend liviano

La imagen `web` usa un *build* multi-stage con `output: standalone`, resultando en ~270 MB. La URL del API (`NEXT_PUBLIC_API_URL`) se fija como *build-arg* (por defecto `http://localhost:8000`); el navegador del host habla con el backend publicado, sin DNS entre contenedores.

8 Completar el recomendador en Docker

El primer despliegue en Docker reveló que el recomendador fallaba con un `FileNotFoundError` sobre `models/baseline_scores.pkl`. Se resolvió con:

1. `src/baseline.py` y la etapa `baseline` del pipeline, que generan el artefacto faltante con la fórmula exacta del notebook 03.
2. Motor (`engine.py`) **resiliente** a la ausencia de `links.csv` (parte del dataset crudo, no versionada): si falta, los pósters usan arte procedural y el recomendador funciona igual.
3. El *entrypoint* genera el baseline en el primer arranque.

✓ Verificado

El motor carga con y sin `links.csv`, y el endpoint que fallaba (`/api/recommendations/home/guest`) responde HTTP 200.

9 Validación extremo a extremo con Docker

Con el daemon de Docker operativo (Server 29.5.3, Compose v5.1.4) se validó el flujo completo:

- **Build:** imágenes `omnirec-backend` (6.25 GB) y `omnirec-web` (270 MB).
- **Arranque limpio** (`down -v` → `up`): el *entrypoint* construyó descriptores, índice FAISS, SVD (4.6 s) y baseline sin intervención. Log: *“Motor de recomendación cargado en 0.3s (55 113 películas, 30 701 con SVD)”*.
- **Endpoints (todos HTTP 200):** `/api/health`, búsqueda semántica con evidencia (*Memento*, *Eternal Sunshine*, *Repulsion*), RAG generativo, `/api/recommendations/home/guest` (el que fallaba) y recomendaciones *folding-in* (*Fight Club*, *Pulp Fiction*, *Shawshank*).
- **Frontend:** `/` y `/buscar` sirven correctamente y CORS permite al navegador hablar con el backend.
- **Los 4 servicios** operativos; el loop MLOps (`evaluate` → `MLflow`, `register` → `SHA256`, `monitor`) corre dentro del contenedor.

⚠ Incidencia de memoria (OOM) resuelta

El primer entrenamiento en contenedor murió con Killed (OOM) porque `COPY data/` incluía `ratings_prepared_60pct.parquet` (15 M de filas, presente en disco pero no versionado). Se **excluyó** ese archivo en `.dockerignore`, de modo que la imagen use `ratings_knn_10pct` (2.5 M), igual que un clon limpio. Tras el ajuste, el entrenamiento corre en 4.6 s sin OOM.

10 Diagnóstico de MLflow

Ante el reporte de que `localhost:5000` “no mostraba nada”, se diagnosticó que **el servicio no estaba corriendo** (no era un problema de configuración):

1. `docker compose down` detiene todos los servicios.
2. `mlflow` y `jupyter` están detrás de **perfiles**, por lo que un `docker compose up` normal no los levanta.
3. Contenedores **zombi** de una sesión previa (`mlflow` salió con código 137, `jupyter` con 128) dejaron una red de Docker corrupta que impedía recrearlos. Se eliminaron y se purgó la red.

Tras levantar con el perfil correcto, se verificó que MLflow sirve la UI (`<title>MLflow</title>`) y contiene el experimento `omnirec-fase4-mlops-rag` con su `run` y métricas. Comando correcto:

```
docker compose --profile tools up -d mlflow # o: make mlflow
```

11 Higiene del repositorio: .gitignore

Se reescribió el `.gitignore` de forma cuidadosa y se **verificó con** `git check-ignore`. Cubre: secretos (`.env*`, conservando los `.env.example`), bases de datos locales (`*.db`, `*.sqlite*`), modelos y artefactos (`models/*`), tracking (`mlflow.db`, `mlruns/`), reportes generados (`reports/monitoring/`), datos regenerables (`descriptors.parquet`, `ratings_prepared_60pct.parquet`), entornos y cachés, y `cruft` de OS/editor.

⚠ Error real detectado y corregido

Los **comentarios en la misma línea de un patrón no son válidos** en `.gitignore` (quedan como parte del patrón), por lo que tres reglas no estaban ignorando lo que debían; se movieron los comentarios a su propia línea. Se confirmó que ningún archivo necesario (los `parquets` que el Docker requiere, ni el entregable) quedó ignorado.

12 Inventario completo de archivos

12.1 Archivos creados

Archivo	Propósito
<code>src/__init__.py</code>	Paquete de la Fase 4
<code>src/utils.py</code>	Logging, UTF-8 en Windows, hashing SHA256
<code>src/config.py</code>	Carga de YAML y resolución de rutas
<code>src/data.py</code>	Carga/validación de los <code>parquets</code> de entrada
<code>src/descriptors.py</code>	Descriptores textuales por película
<code>src/embeddings.py</code>	Encoder Sentence-Transformers
<code>src/index.py</code>	Índice FAISS + metadatos + embeddings de tags

Archivo	Propósito
src/search.py	Recuperación con re-ranking y evidencia
src/generate.py	Capa RAG generativa (LLM + fallback)
src/train.py	Reentrenamiento del SVD
src/baseline.py	Baseline de popularidad bayesiana
src/registry.py	Versionado de artefactos (SHA256)
src/tracking.py	Wrapper de MLflow
src/evaluate.py	Métricas recomendador + recuperación
src/monitor.py	Monitoreo de calidad/drift
src/pipeline.py	Orquestador CLI por etapas
config/pipeline.yaml	Parámetros del pipeline y SVD
config/rag.yaml	Parámetros de RAG/recuperación
config/monitoring.yaml	Umbrales y política de reentrenamiento
app/backend/ml/semantic.py	Puente backend ↔ src
app/backend/routers/search.py	Endpoint de búsqueda semántica
app/web-app/app/buscar/page.tsx	Página de búsqueda en OmniCine
app/web-app/components/SearchResultCard.tsx	Tarjeta de resultado con evidencia
docker-compose.yml	Orquestación de los 4 servicios
app/backend/Dockerfile	Imagen Python + stack ML
app/backend/docker-entrypoint.sh	Construye artefactos y arranca
app/web-app/Dockerfile	Imagen Next.js standalone (multi-stage)
.dockerignore	Contexto de build liviano (raíz)
app/web-app/.dockerignore	Contexto de build del frontend
.env.example	Variables (clave LLM, puertos)
Makefile	Atajos de Docker
reports/PLAN_FASE4_MLOPS_RAG.md	Plan de implementación
reports/INFORME_FASE4_MLOPS_RAG.md	Informe de la fase
reports/figures/fase4_*.png	Figuras (cobertura, t-SNE)

12.2 Archivos modificados

Archivo	Cambio
app/backend/main.py	Registro del router <code>search</code> ; arranque resiliente
app/backend/ml/engine.py	Tolerancia a <code>links.csv</code> ausente
app/web-app/lib/types.ts	Tipos <code>SemanticResult/SemanticResponse</code>
app/web-app/lib/api.ts	Método <code>semanticSearch</code>
app/web-app/components/Navbar.tsx	Enlace “Buscar”; lupa → <code>/buscar</code>
app/web-app/next.config.ts	<code>output: "standalone"</code>
notebooks/05_Semantic_Search_RAG.ipynb	Reescrito y ejecutado

Archivo	Cambio
requirements.txt	Dependencias Fase 4 + torch==2.4.*
.gitignore	Reescritura cuidadosa y verificada
README.md	Sección Docker + estado Fase 4
src/README.md, config/README.md, models/README.md	Documentación de carpetas

13 Resultados y figuras

La Figura 1 muestra la cobertura de *genome-scores* del catálogo; la Figura 2 muestra la proyección t-SNE de los embeddings coloreada por género principal, evidenciando que el espacio latente captura semántica útil.

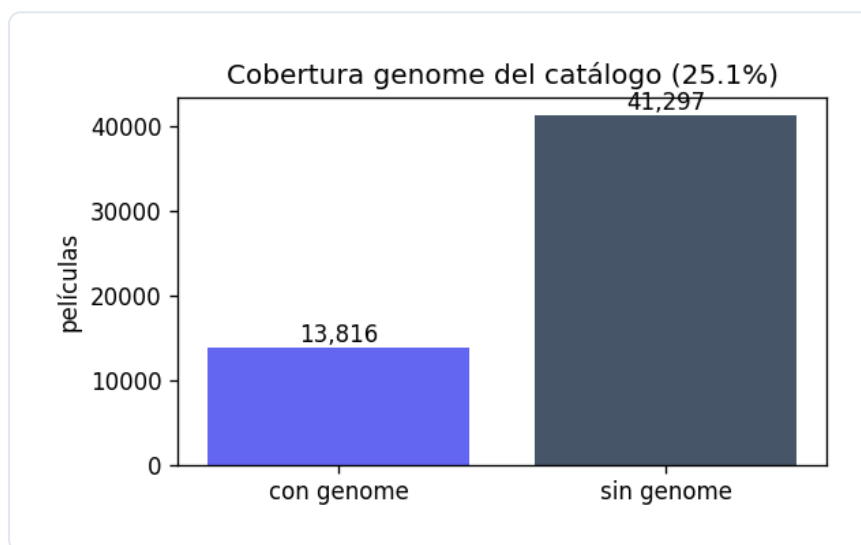


Figura 1: Cobertura de *genome-scores* del catálogo (25.1 %).

i Ejemplo de consulta documentada

Para “thriller psicológico sobre la memoria” el sistema recupera *Memento* (evidencia: *storytelling, memory, memory loss*), *Eternal Sunshine of the Spotless Mind* y *Repulsion*, demostrando recuperación pertinente y explicable.

14 Cómo ejecutar

14.1 Con Docker (recomendado)

```
git clone <repo> && cd OmniRec-Movies
cp .env.example .env # opcional: clave Claude para la capa
                        generativa
docker compose up --build # -> http://localhost:3000/buscar
docker compose --profile tools up -d mlflow # MLflow -> :5000
docker compose --profile notebooks up -d jupyter # Jupyter -> :8888
```

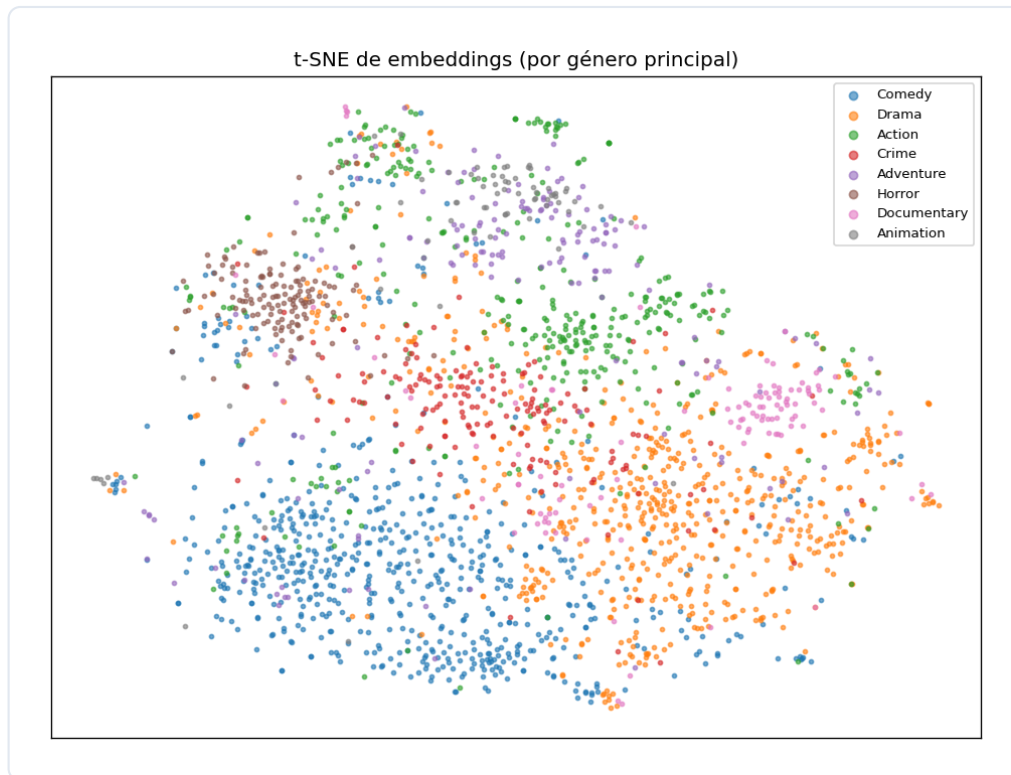


Figura 2: t-SNE de los embeddings de descriptores, por género principal.

14.2 Nativo (sin Docker)

```
pip install -r requirements.txt
python -m src.pipeline all          # entrena, indexa, evalúa, versiona
python -m src.pipeline monitor     # reporte de monitoreo
```

15 Conclusiones y trabajo futuro

Se entregó un componente de **recuperación semántica multilingüe** con evidencia y capa generativa, y un **flujo MLOps mínimo viable** (pipeline reproducible, tracking, versionado y monitoreo), todo **dockerizado** para una inicialización de un solo comando. El recomendador quedó completo y la plataforma validada extremo a extremo.

⚠ Limitaciones

La cobertura de *genome* (~25 %) limita la recuperación de la *long tail*; el re-ranking por popularidad introduce un sesgo hacia títulos *mainstream* (parámetro controlable); los *tags* están en inglés (mitigado por el encoder multilingüe). La imagen del backend es grande (~6.25 GB), optimizable con un *build* multi-stage.

📌 Trabajo futuro

Optimización del tamaño de imagen; incorporar `links.csv` para pósters reales; ampliar el monitoreo con métricas en línea; y, opcionalmente, dejar MLflow/Jupyter activos por defecto.

📺 Fin del informe técnico de cambios • OmniRec-Movies • Grupo 3 · Vincit Data